

離散制御器合成における計算空間爆発 抑制のための段階的なLTSの最小化

山口友輝

東京科学大学 鄭研究室

離散事象システム

離散事象システム^[1]

- アクションの発火によって、状態が離散的に遷移するシステム
- LTS^[1]（ラベル付き状態遷移システム）で表現が可能

安全性が保証された離散事象システム

- **安全性**：システムにとって悪い事象が発生しないこと
- システムが制御不能な事象が発生しようとも悪い状態が発生しないようにする
→安全性が保証されているということ
- セキュリティなどのミス（悪い事象の発生）が許されないシステムで
安全性の保証が特に有効である

[1] Jeff Magee and Jeff Kramer. Concurrency: State Models and Java Programs.

安全性が保証された離散事象システム

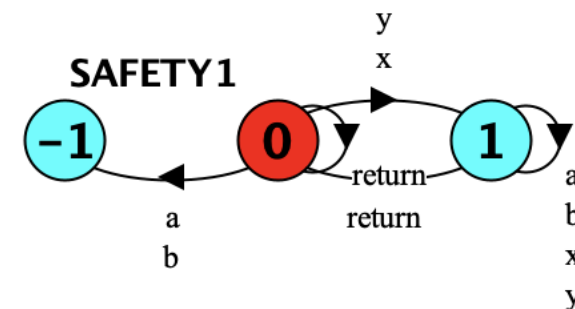
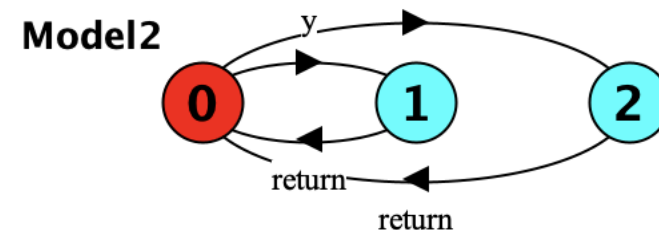
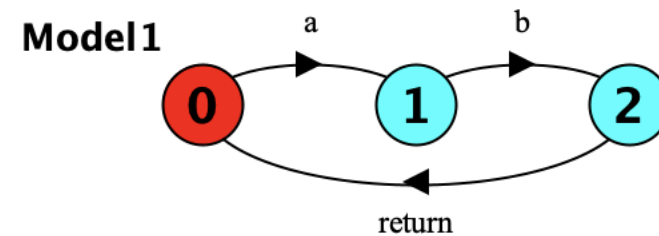
安全性が保証された離散事象システムの設計者は
まず、**環境モデル**と**監視モデル**を用意する

環境モデル^[1]

- システムの動作環境をLTSとして表現したもの

監視モデル^[1]

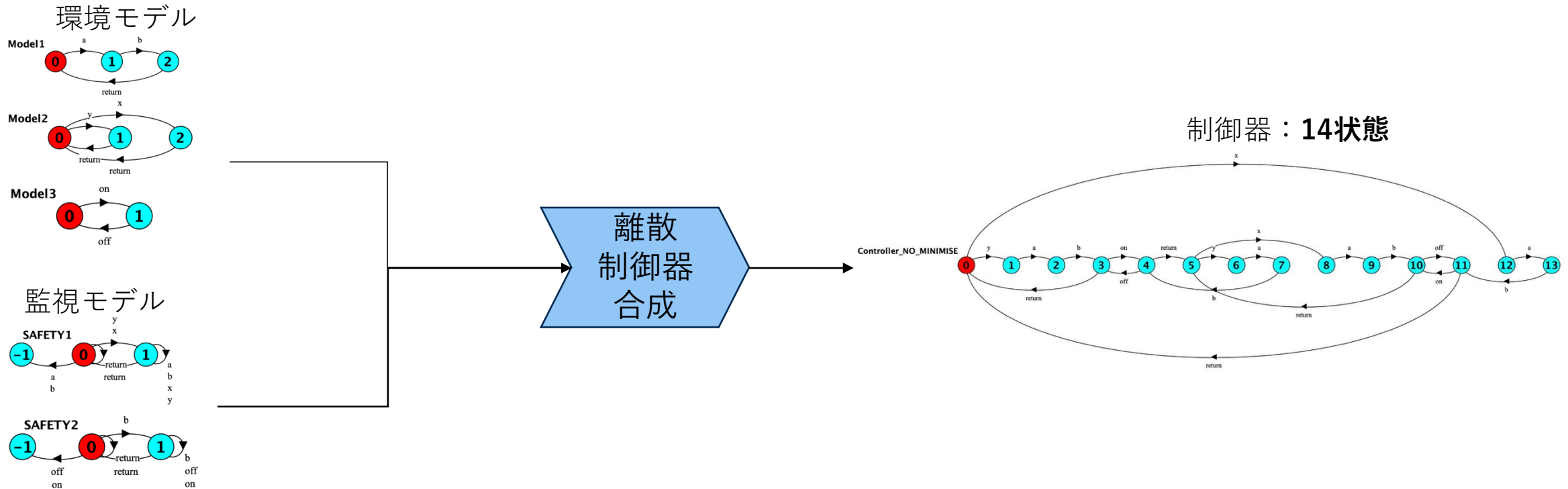
- システムの安全性をLTSとして表現したもの
- 安全性を違反する事象が発生したときに違反状態（-1の状態）に遷移させることで安全性を表現



[1] N. R. D'Ippolito, et.al., "Synthesis of live behaviour models," FSE '10

離散制御器合成^[1]

- 環境モデルと監視モデルを入力として、これらを自動で合成し、**制御器**を出力する
- 制御器は監視モデルの安全性を保証する動作仕様としてLTSで表現される
- 離散制御器合成は並列合成と安全性分析の2つのプロセスで行われる

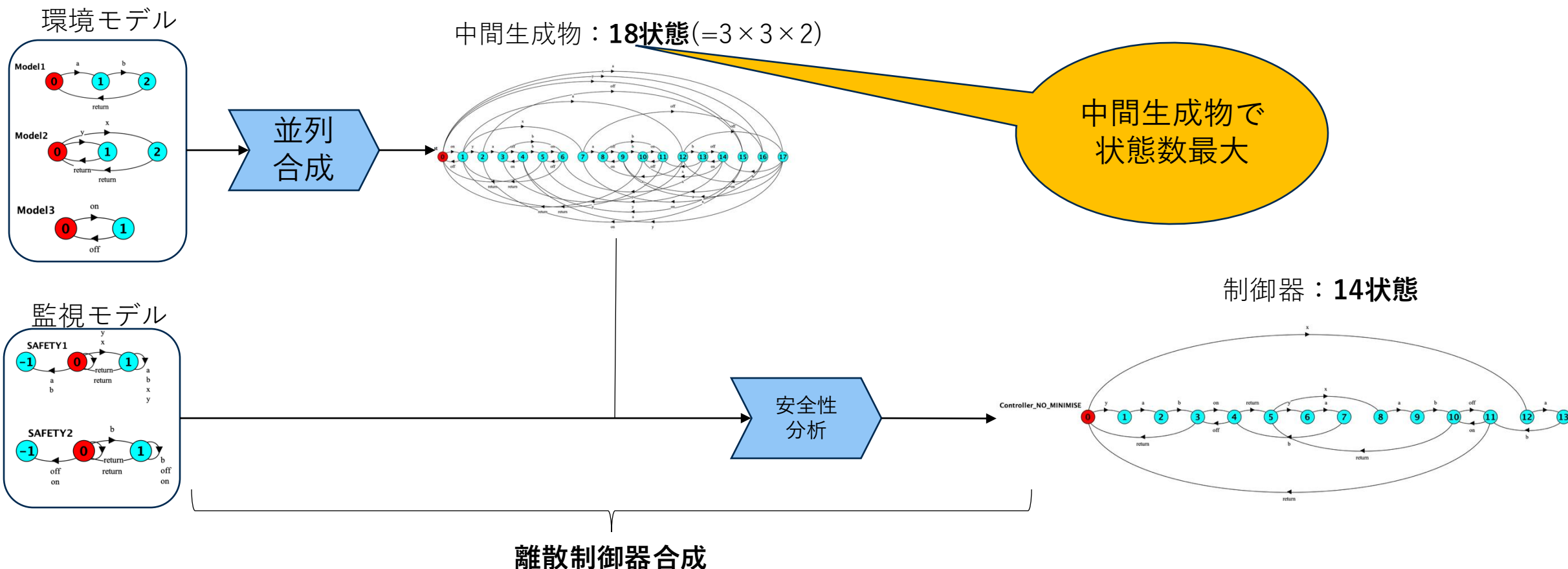


[1] N. R. D'Ippolito, et.al., "Synthesis of live behaviour models," FSE '10

離散制御器合成の課題

計算空間の状態数は入力モデルの数に対して指数増加する

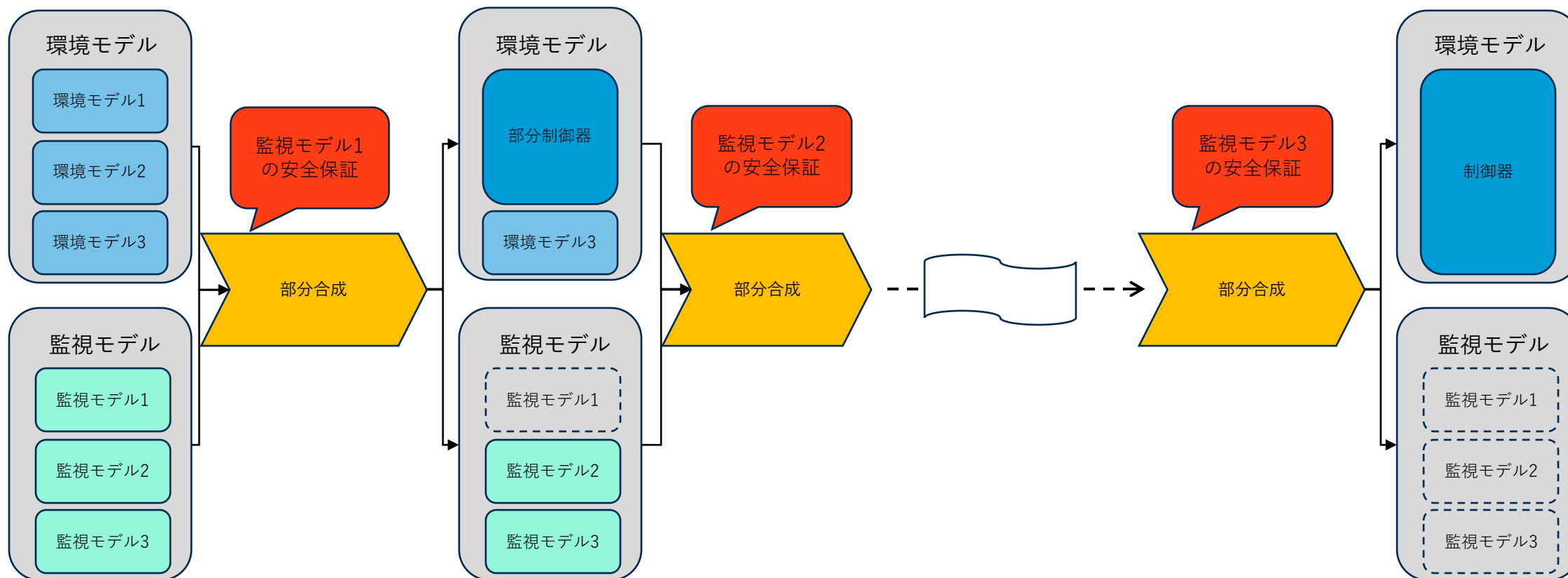
- 合成の際に構築される状態空間は各入力モデルの状態数の積によって決まる
- 大規模なシステムでは適用できない
- 特に、離散制御器合成の際に生成される**状態数が最大となる中間生成物**がボトルネックになる



従来手法：段階的制御器合成

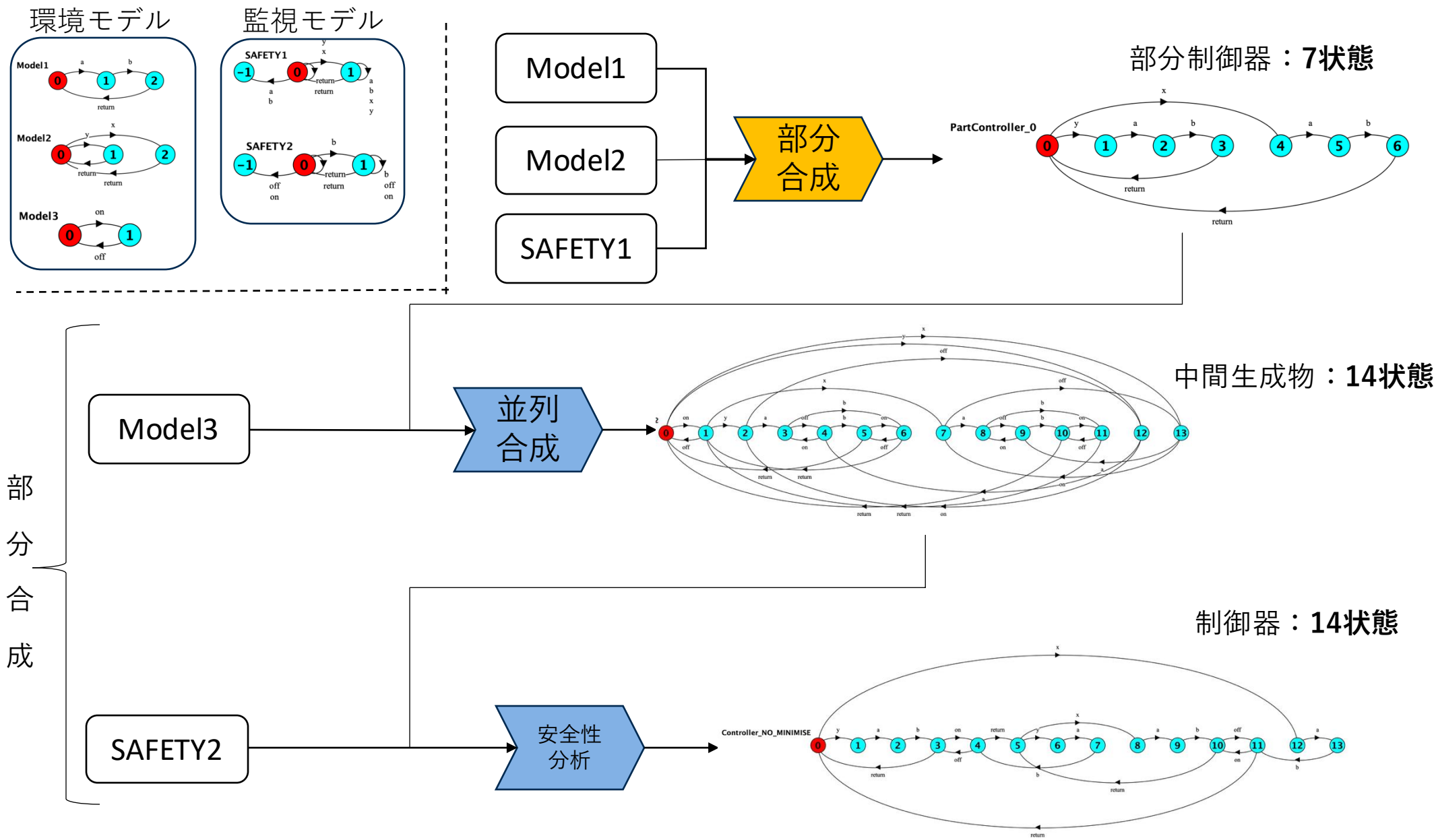
段階的制御器合成^[1]

- 段階的に安全性を保証して制御器を合成する。
- 監視モデルごとに部分制御器を合成し，その部分制御器を以降入力とし，新たな部分制御器を合成する。
これを，全ての監視モデルが環境モデルに反映されるまで行う。



[1]山内 拓人, et. al.: 段階的な部分合成による離散制御器合成の分析空間削減, 電子情報通信学会論文誌 D (2023)

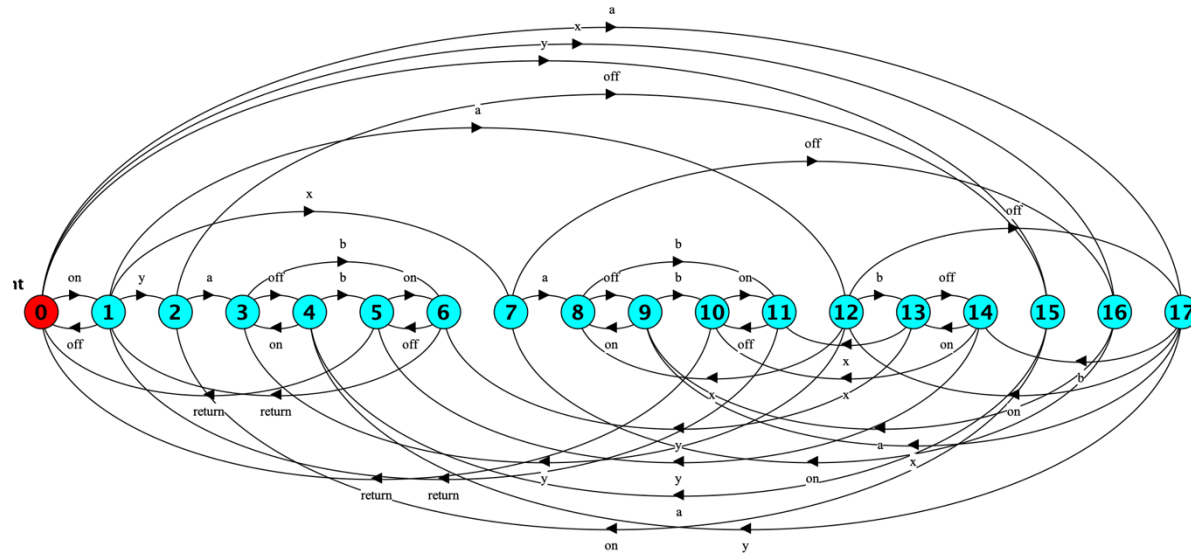
従来手法：段階的制御器合成



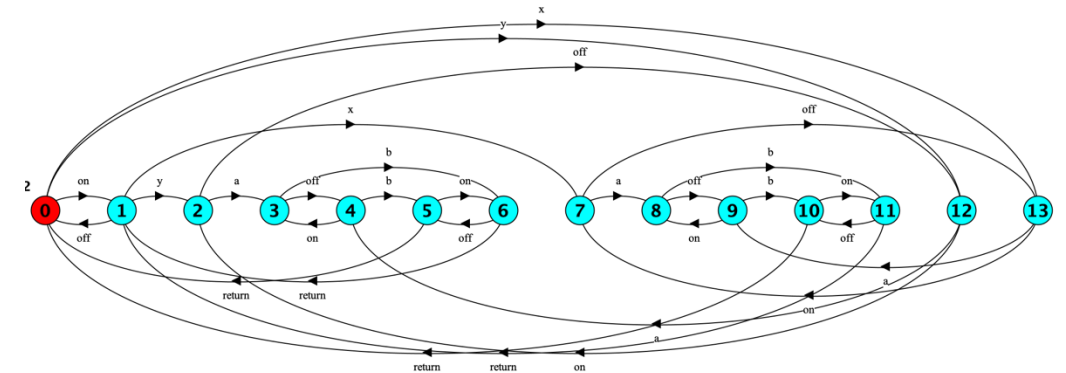
従来手法：段階的制御器合成（14状態だと 不十分という吹き出し）

段階的制御器合成で生成される中間生成物

- 分析する環境モデルに対し，最低限の環境モデルを合成に用いることで，離散制御器合成のときより小さい中間生成物が生成される



通常の離散制御器合成の中間生成物：18状態

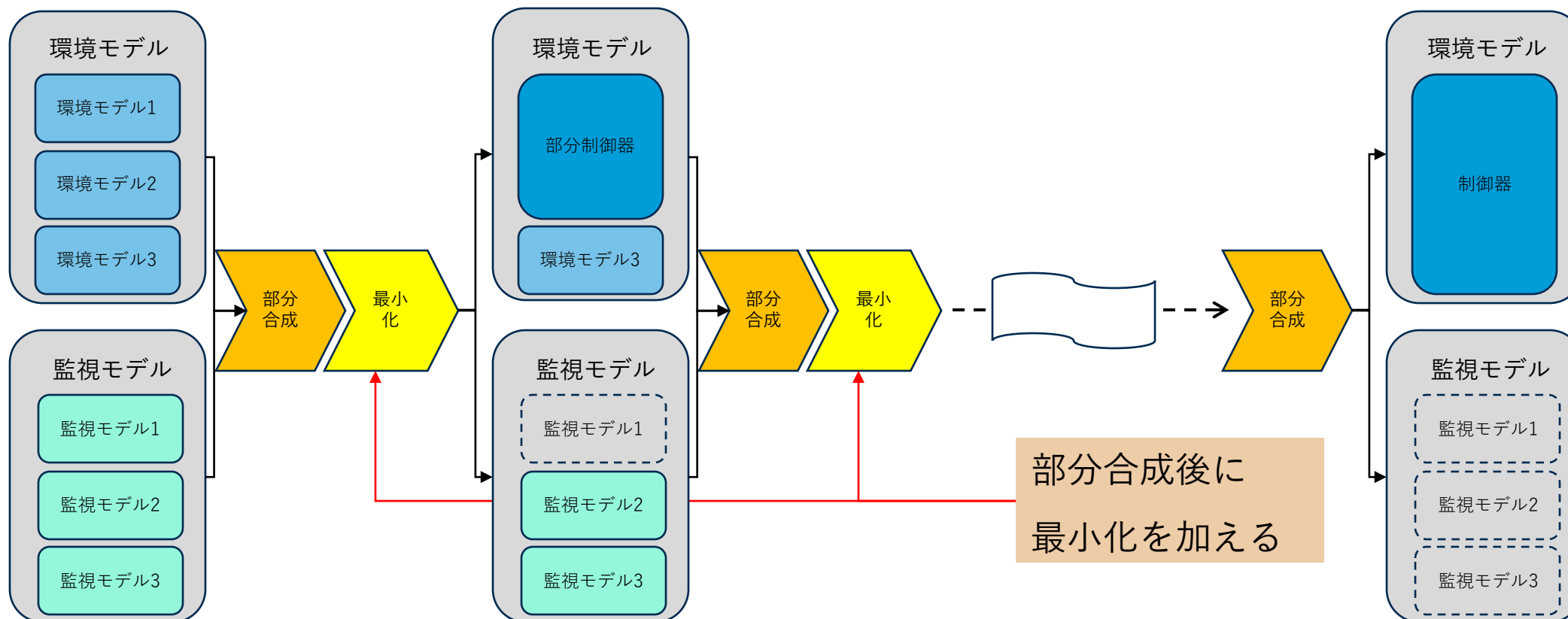


段階的制御器合成の中間生成物：14状態

提案手法：段階的制御器合成 + LTSの最小化

段階的制御器合成の各段階の最後に毎回最小化する手法を提案

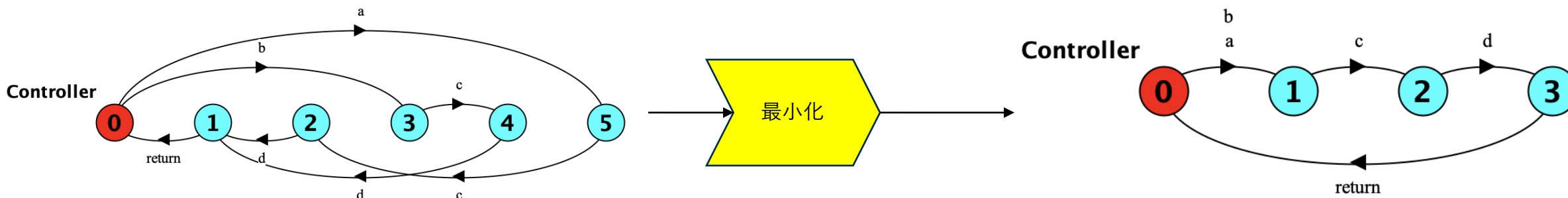
- 部分制御器ごとに最小化を適用することで、各合成段階での状態爆発を抑制
- **LTSの等価性**：ある 2 つの LTS から生成されるすべての事象列が同じときに、その LTS が等価とみなせる性質
- **LTSの最小化**^[1]：**LTSの等価性**に基づいて、LTSを状態数が最小のものに変換する操作



[1] Jeff Magee and Jeff Kramer. Concurrency: State Models and Java Programs.

提案手法の説明（LTSの最小化）：例1

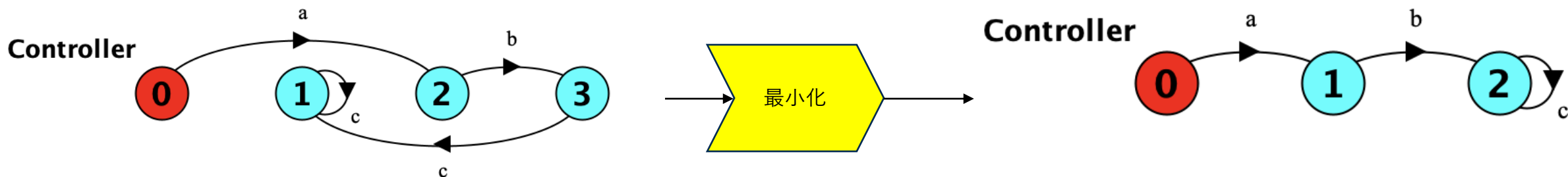
LTSの等価性を保ったまま状態数を最小化する



- アクションa, bが発火した後は, c, d, returnを順に発火し状態0に戻るものでどちらを選択しても同じ.
- 最小化によって, アクションa, bを発火したときに生成される状態が1つにまとめられ, 状態数が削減された.

提案手法の説明（LTSの最小化）：例2

LTSの等価性を保ったまま状態数を最小化する

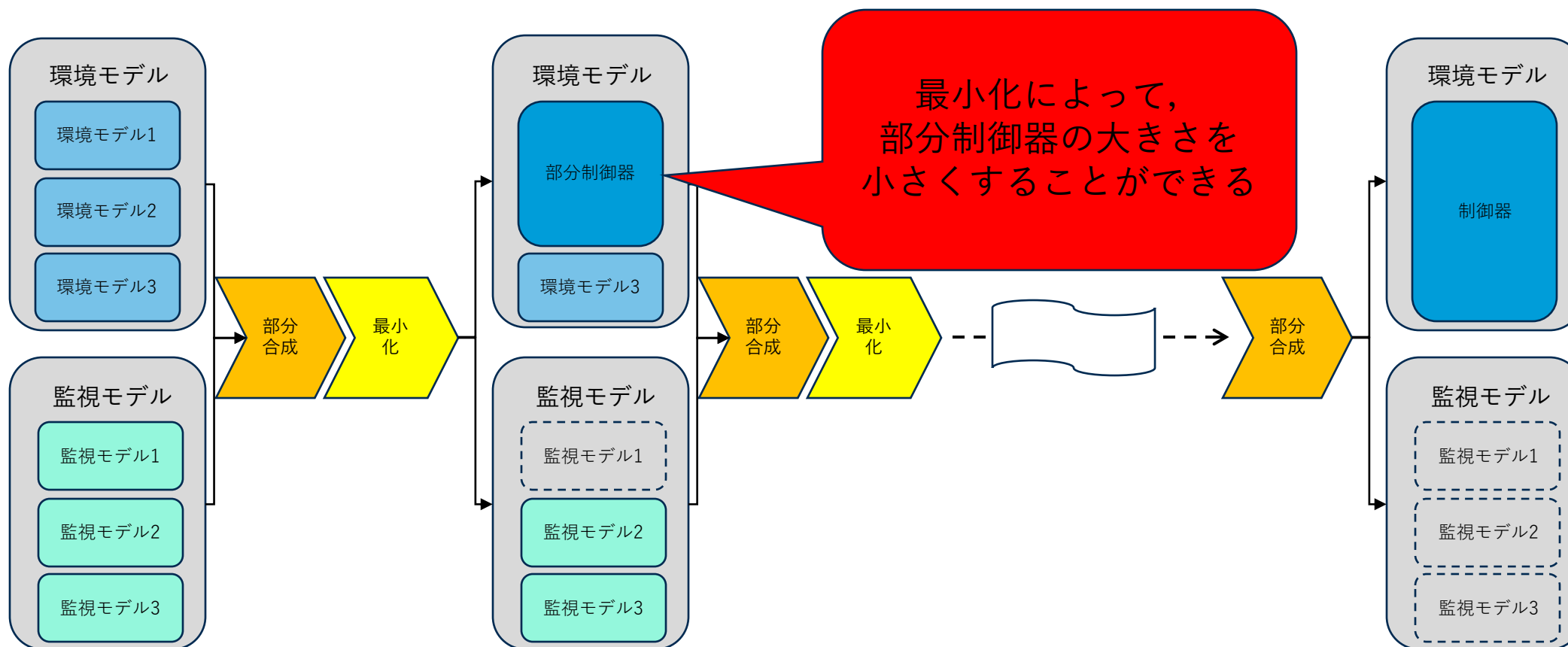


- 最終的にアクションcを発火するループに入る制御器
- ループの直前にアクションcの発火による冗長な状態と遷移を追加すると、最小化により、それらが削減される

提案手法：段階的制御器合成 + LTSの最小化

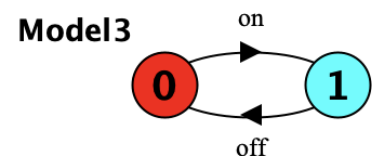
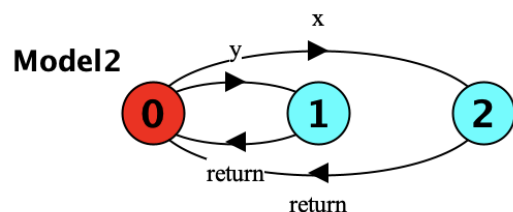
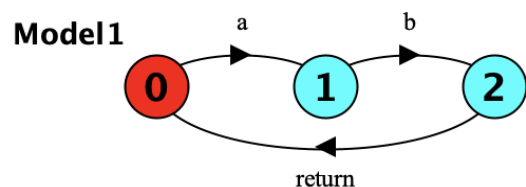
段階的制御器合成の各段階の最後に毎回最小化する手法を提案

- 部分制御器ごとに最小化を適用することで、各合成段階での状態爆発を抑制
- **LTSの等価生**：ある 2 つの LTS から生成されるすべての事象列が同じときに、その LTS が等価とみなせる性質
- **LTSの最小化**：LTSの等価性に基づいて、LTSを状態数が最小のものに変換する操作

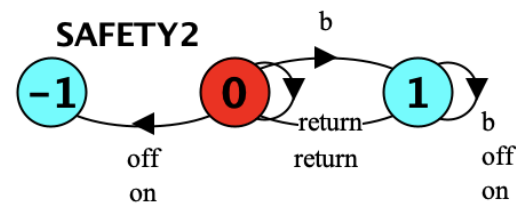
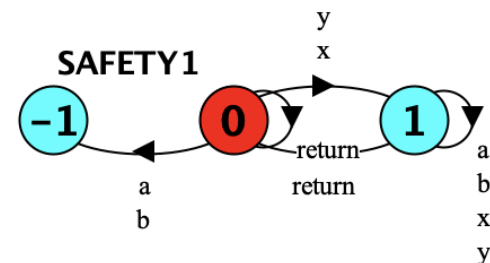


提案手法の例

段階的に合成と最小化を適用したときの例を示す



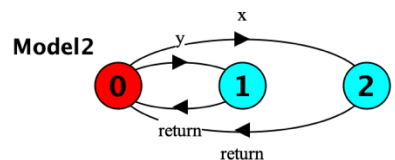
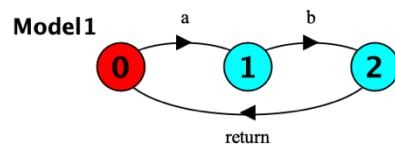
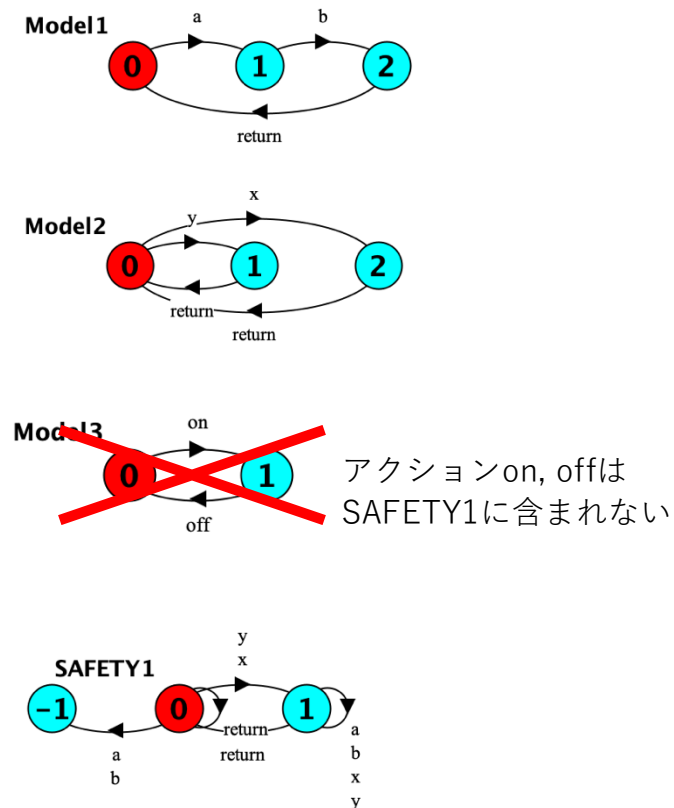
環境モデル



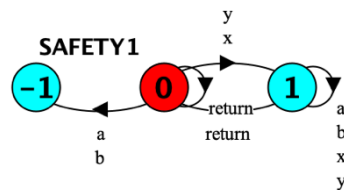
監視モデル

提案手法の例

監視モデルSAFETY1の安全性保証

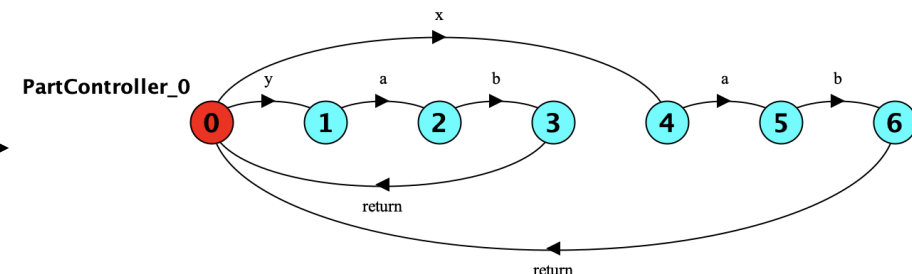


環境モデル



監視モデル

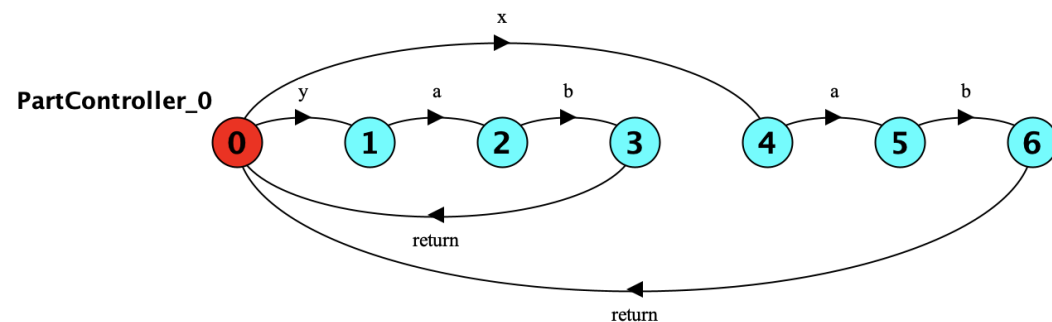
部分
合成



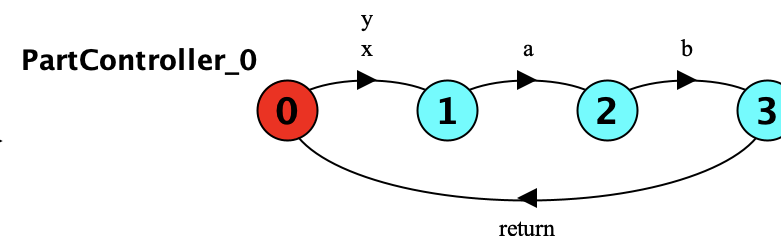
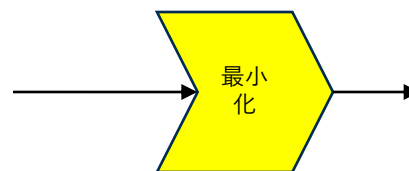
最小化する前の部分制御器：7状態

提案手法の例

PartController_0の最小化



7状態

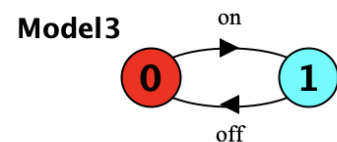
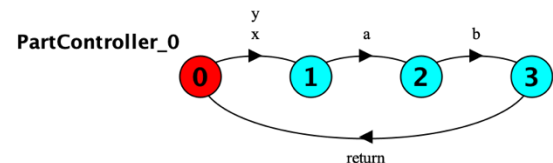


4状態

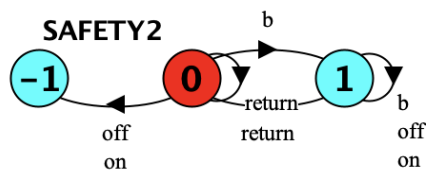
最小化によって、状態数を削減

提案手法の例

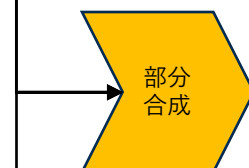
監視モデルSAFETY2の安全性保証



環境モデル

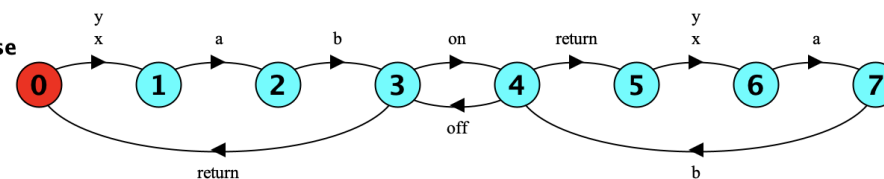


監視モデル

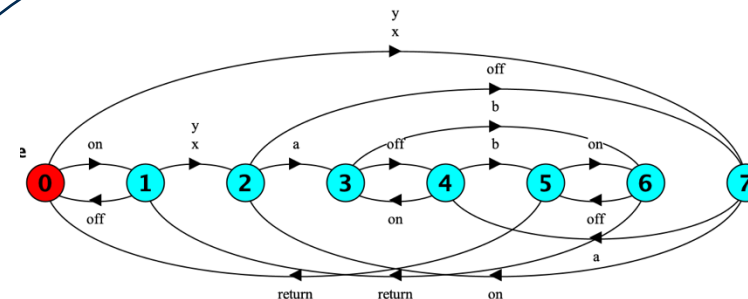


部分
合成

Controller_Minimise



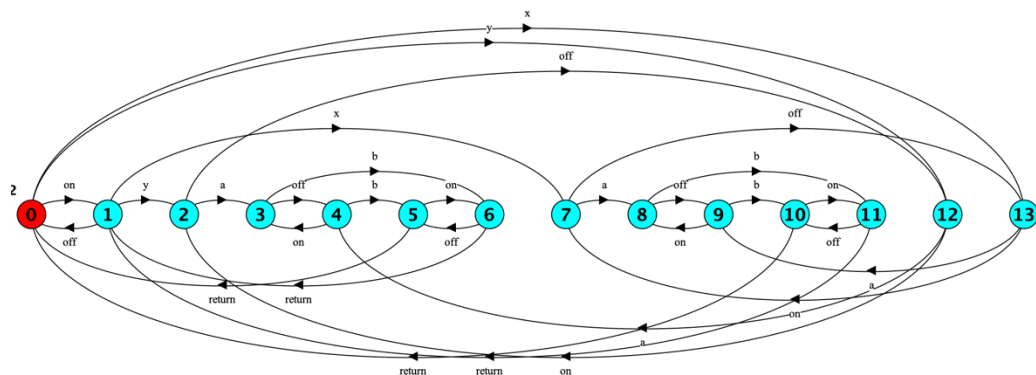
制御器：8状態



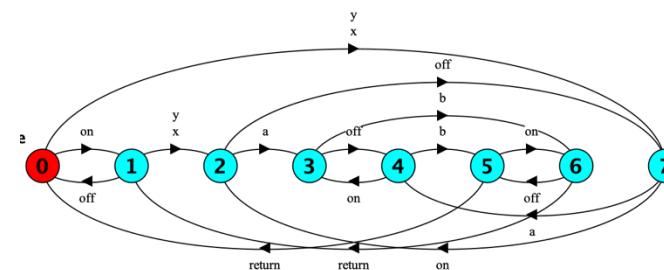
中間生成物：8状態

提案手法の例

最小化を導入していないときの比較



従来手法の中間生成物：14状態



提案手法の中間生成物：8状態

最小化によって、中間生成物の状態数が削減されていることがわかる。

評価：Research Question

提案手法の効果を確認するために、以下のResearch Questionを設定し検証

RQ1. LTSの最小化を加えたことで、状態数と遷移数がどれほど削減されたか

RQ2. LTSの最小化を加えたことで、主記憶量と計算時間がどれほど削減されたか

以上のRQを検証するために、従来の段階的制御器合成と、段階的制御器合成にLTSの最小化を適用した手法で制御器合成し、状態数、遷移数、主記憶量、合成時間、最小化時間を測定する実験を行った。

評価：実験設定

実験に使用したシステム

システムのスケールを変更可能な以下の2つのシステムで実験を行った

- 人数管理システム^[1]：複数の部屋の人数を管理するシステム
- 自動倉庫管理システム^[2]：複数台のロボットで棚の在庫を管理するシステム

使用した計算機

OS：Windows 11 Pro

CPU：Intel Core Ultra 9 285K

RAM：128GB

使用した制御器合成ツール

Model Transition Synsystem Analyzer(MTSA)^[3]

[1] A. Borda, L. Pasquale, V. Koutavas, and B. Nuseibeh, “Compositional verification of self-adaptive cyber-physical systems,” 2018IEEE/ACM 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp.1–11, 2018.

[2] J. Enright and P.R. Wurman, “Optimization and coordinated au-tonomy in mobile fulfillment systems,” Automated Action Plan-ning for Autonomous Mobile Robots, 2011 AAAI Workshop, SanFrancisco, California, USA, August 7, 2011, pp.33–38, 2011. <http://www.aaai.org/ocs/index.php/WS/AAAIW11/paper/view/3917>

[3] N. D'Ippolito, D. Fischbein, M. Chechik, and S. Uchitel, “MTSA: the modal transition system analyser,” 23rd IEEE/ACMInternational Conference on Automated Software Engineering,15-19 September 2008, L'Aquila, Italy, pp.475–476, 2008. <https://doi.org/10.1109/ASE.2008.78>

評価：実験結果

人数管理システム：Nは部屋の数，Kは人の数

自動倉庫管理システム：Nはロボットの数，Kは棚の数

計算時間 = 合成時間 + 最小化時間

		最小化なし					最小化あり					
	N	K	状態数	遷移数	主記憶量 (MB)	合成 (計算) 時間 (ms)	状態数	遷移数	主記憶量 (MB)	合成時間 (ms)	最小化時間 (ms)	計算時間 (ms)
人数管理	2	4	211	372	45.5	354	188(-10.9%)	337(-9.4%)	61.9(+36.1%)	417(+17.8%)	2	419(+18.4%)
	3	4	2173	3363	48.7	718	1830(-15.8%)	2850(-15.3%)	62.0(+27.3%)	703(-2.1%)	50	753(+4.9%)
	4	4	5953	10580	74.5	1420	4059(-31.8%)	7425(-29.8%)	67.2(-9.7%)	1204(-15.2%)	301	1505(+6.0%)
	5	4	15725	27814	124.9	2614	12350(-21.5%)	22603(-18.7%)	127.0(+1.7%)	2111(-19.2%)	1807	3918(+50.0%)
	6	4	60131	106816	375.6	6354	46172(-23.2%)	84865(-20.6%)	538.5(+43.3%)	5145(-19.0%)	28909	34054(+436.1%)
	7	4	223589	398443	1334.5	24451	168590(-24.6%)	310963(-22.0%)	4628.5(+246.8%)	17522(-28.3%)	432659	450181(+1741.4%)
	8	4	813323	1453630	4995.0	186465	604100(-25.7%)	1117561(-23.1%)	49809.9(+898.8%)	110331(-40.8%)	6640130	6750461(+3520.3%)
	9	4	777602	1392394	4629.3	167493	568379(-26.9%)	1056325(-24.1%)	44468.5(+860.5%)	96260(-42.5%)	5552781	5649041(+3270.0%)
	5	5	19037	31808	140.4	2727	14960(-21.4%)	25247(-20.6%)	157.3(+12.0%)	2339(-14.2%)	2685	5024(+84.2%)
	5	6	22349	37361	160.2	2782	17570(-21.4%)	29666(-20.6%)	177.6(+10.9%)	2441(-12.3%)	3907	6348(+128.3%)
	5	7	25661	42914	189.2	2758	20180(-21.4%)	34085(-20.6%)	210.9(+11.4%)	2637(-4.4%)	5241	7878(+185.7%)
	5	8	28973	48467	202.6	2963	22790(-21.3%)	38504(-20.6%)	235.8(+16.4%)	2661(-10.2%)	6536	9197(+210.5%)
	5	9	32285	54020	229.4	3037	25400(-21.3%)	42923(-20.5%)	269.2(+17.3%)	2861(-5.8%)	8243	11104(+265.6%)
	5	10	35597	59573	243.7	3271	28010(-21.3%)	47342(-20.5%)	307.6(+26.2%)	2582(-21.1%)	7859	10441(+219.3%)
自動倉庫	2	2	6400	47438	124.2	1874	4832(-24.5%)	35408(-25.4%)	98.8(-20.4%)	1716(-8.4%)	7	1723(-8.0%)
	3	2	489212	2100488	4183.5	37451	115348(-76.4%)	506768(-75.9%)	1082.6(-74.1%)	10463(-72.1%)	22038	32501(-13.2%)
	4	2	670320	3000732	6255.2	95748	228228(-66.0%)	1006985(-66.4%)	2144.9(-65.7%)	20288(-78.8%)	1392	21680(-77.4%)
	2	5	19807	195902	348.0	3060	11024(-44.3%)	106502(-45.6%)	209.5(-39.8%)	2407(-21.3%)	109	2516(-17.8%)

評価：実験結果（RQ1の評価）

人数管理システム：Nは部屋の数，Kは人の数

自動倉庫管理システム：Nはロボットの数，Kは棚の数

計算時間 = 合成時間 + 最小化時間

削減率の平均

人数管理
状態数：-21.9%
遷移数：-19.9%

自動倉庫管理
状態数：-52.8%
遷移数：-53.3%

最小化なし												
	N	K	状態数	遷移数	主記憶量 (MB)	合成 (計算) 時間 (ms)	状態数	遷移数	主記憶量 (MB)	合成時間 (ms)	最小化時間 (ms)	計算時間 (ms)
人数管理	2	4	211	372	45.5	354	188(-10.9%)	337(-9.4%)	45.5	417(+17.8%)	2	419(+18.4%)
	3	4	2173	3363	48.7	718	1830(-15.8%)	2850(-15.3%)	48.7	703(-2.1%)	50	753(+4.9%)
	4	4	5953	10580	74.5	1420	4059(-31.8%)	7425(-29.8%)	74.5	1204(-15.2%)	301	1505(+6.0%)
	5	4	15725	27814	124.9	2614	12350(-21.5%)	22603(-18.7%)	124.9	2111(-19.2%)	1807	3918(+50.0%)
	6	4	60131	106816	375.6	6354	46172(-23.2%)	84865(-20.6%)	375.6	5145(-19.0%)	28909	34054(+436.1%)
	7	4	223589	398443	1334.5	24451	168590(-24.6%)	310963(-22.0%)	1334.5	17522(-28.3%)	432659	450181(+1741.4%)
	8	4	813323	1453630	4995.0	186465	604100(-25.7%)	1117561(-23.1%)	4995.0	110331(-40.8%)	6640130	6750461(+3520.3%)
	9	4	777602	1392394	4629.3	167493	568379(-26.9%)	1056325(-24.1%)	4629.3	96260(-42.5%)	5552781	5649041(+3270.0%)
	5	5	19037	31808	140.4	2727	14960(-21.4%)	25247(-20.6%)	140.4	2339(-14.2%)	2685	5024(+84.2%)
	5	6	22349	37361	160.2	2782	17570(-21.4%)	29666(-20.6%)	160.2	2441(-12.3%)	3907	6348(+128.3%)
	5	7	25661	42914	189.2	2758	20180(-21.4%)	34085(-20.6%)	189.2	2637(-4.4%)	5241	7878(+185.7%)
	5	8	28973	48467	202.6	2963	22790(-21.3%)	38504(-20.6%)	202.6	2661(-10.2%)	6536	9197(+210.5%)
	5	9	32285	54020	229.4	3037	25400(-21.3%)	42923(-20.5%)	229.4	2861(-5.8%)	8243	11104(+265.6%)
	5	10	35597	59573	243.7	3271	28010(-21.3%)	47342(-20.5%)	243.7	2582(-21.1%)	7859	10441(+219.3%)
自動倉庫	2	2	6400	47438	124.2	1874	4832(-24.5%)	35408(-25.4%)	124.2	1716(-8.4%)	7	1723(-8.0%)
	3	2	489212	2100488	4183.5	37451	115348(-76.4%)	506768(-75.9%)	4183.5	10463(-72.1%)	22038	32501(-13.2%)
	4	2	670320	3000732	6255.2	95748	228228(-66.0%)	1006985(-66.4%)	6255.2	20288(-78.8%)	1392	21680(-77.4%)
	2	5	19807	195902	348.0	3060	11024(-44.3%)	106502(-45.6%)	348.0	2407(-21.3%)	109	2516(-17.8%)

RQ1. LTSの最小化を加えたことで，状態数と遷移数がどれほど削減されたか

→いずれのシステムでも平均して状態数と遷移数が削減されていることがわかった。

また，人数管理システム，自動倉庫管理システムのどちらでもN,Kを変えても削減効果はさほど変わらず，削減効果の変化に相関はなかった。

評価：実験結果（RQ1の評価）

1つのシステムの計算空間の詳細

人数管理システムのN=6, K=4の場合に各部分合成での合成と最小化の際の状態数と時間を計測

	最小化なし		最小化あり	
	状態数	時間 (ms)	状態数	時間 (ms)
1 回目合成	67	180	67	209
1 回目最小化	-	-	55	0
2 回目合成	107	82	92	62
2 回目最小化	-	-	83	0
3 回目合成	240	104	192	78
3 回目最小化	-	-	192	1
4 回目合成	368	107	307	86
4 回目最小化	-	-	281	3
5 回目合成	811	183	637	113
5 回目最小化	-	-	637	4
6 回目合成	1229	164	1004	170
6 回目最小化	-	-	929	13
7 回目合成	2674	234	2074	197
7 回目最小化	-	-	2074	39
8 回目合成	4014	319	3231	238
8 回目最小化	-	-	3015	108
9 回目合成	8649	584	6651	408
9 回目最小化	-	-	6651	334
10 回目合成	12879	711	10260	564
10 回目最小化	-	-	9639	1305
11 回目合成	60131	1946	46172	1576
11 回目最小化	-	-	45198	27096
12 回目合成	39665	1560	31322	1309

各部分合成で最小化することによって状態数が削減され、部分合成による状態数の増加が緩やかになっている

評価：実験結果（RQ2の評価）

人数管理システム：Nは部屋の数，Kは人の数

自動倉庫管理システム：Nはロボットの数，Kは棚の数

計算時間 = 合成時間 + 最小化時間

削減率の平均

人数管理

主記憶量：+168.8%

計算時間：+667.3%

自動倉庫管理

主記憶量：-50.0%

計算時間：-23.3%

		最小化なし						最小化あり					
	N	K	状態数	遷移数	主記憶量 (MB)	合成 (計算) 時間 (ms)	状態数	遷移数	主記憶量 (MB)	合成時間 (ms)	最小化時間 (ms)	計算時間 (ms)	
人数管理	2	4	211	372	45.5	354	188(-10.9%)	337(-9.4%)	61.9(+36.1%)	417(+17.8%)	2	419(+18.4%)	
	3	4	2173	3363	48.7	718	1830(-15.8%)	2850(-15.3%)	62.0(+27.3%)	703(-2.1%)	50	753(+4.9%)	
	4	4	5953	10580	74.5	1420	4059(-31.8%)	7425(-29.8%)	67.2(-9.7%)	1204(-15.2%)	301	1505(+6.0%)	
	5	4	15725	27814	124.9	2614	12350(-21.5%)	22603(-18.7%)	127.0(+1.7%)	2111(-19.2%)	1807	3918(+50.0%)	
	6	4	60131	106816	375.6	6354	46172(-23.2%)	84865(-20.6%)	538.5(+43.3%)	5145(-19.0%)	28909	34054(+436.1%)	
	7	4	223589	398443	1334.5	24451	168590(-24.6%)	310963(-22.0%)	4628.5(+246.8%)	17522(-28.3%)	432659	450181(+1741.4%)	
	8	4	813323	1453630	4995.0	186465	604100(-25.7%)	1117561(-23.1%)	49809.9(+898.8%)	110331(-40.8%)	6640130	6750461(+3520.3%)	
	9	4	777602	1392394	4629.3	167493	568379(-26.9%)	1056325(-24.1%)	44468.5(+860.5%)	96260(-42.5%)	5552781	5649041(+3270.0%)	
	5	5	19037	31808	140.4	2727	14960(-21.4%)	25247(-20.6%)	157.3(+12.0%)	2339(-14.2%)	2685	5024(+84.2%)	
	5	6	22349	37361	160.2	2782	17570(-21.4%)	29666(-20.6%)	177.6(+10.9%)	2441(-12.3%)	3907	6348(+128.3%)	
	5	7	25661	42914	189.2	2758	20180(-21.4%)	34085(-20.6%)	210.9(+11.4%)	2637(-4.4%)	5241	7878(+185.7%)	
	5	8	28973	48467	202.6	2963	22790(-21.3%)	38504(-20.6%)	235.8(+16.4%)	2661(-10.2%)	6536	9197(+210.5%)	
	5	9	32285	54020	229.4	3037	25400(-21.3%)	42923(-20.5%)	269.2(+17.3%)	2861(-5.8%)	8243	11104(+265.6%)	
	5	10	35597	59573	243.7	3271	28010(-21.3%)	47342(-20.5%)	307.6(+26.2%)	2582(-21.1%)	7859	10441(+219.3%)	
自動倉庫	2	2	6400	47438	124.2	1874	4832(-24.5%)	35408(-25.4%)	98.8(-20.4%)	1716(-8.4%)	7	1723(-8.0%)	
	3	2	489212	2100488	4183.5	37451	115348(-76.4%)	506768(-75.9%)	1082.6(-74.1%)	10463(-72.1%)	22038	32501(-13.2%)	
	4	2	670320	3000732	6255.2	95748	228228(-66.0%)	1006985(-66.4%)	2144.9(-65.7%)	20288(-78.8%)	1392	21680(-77.4%)	
	2	5	19807	195902	348.0	3060	11024(-44.3%)	106502(-45.6%)	209.5(-39.8%)	2407(-21.3%)	109	2516(-17.8%)	

RQ2. LTSの最小化を加えたことで，主記憶量と計算時間がどれほど削減されたか

→自動倉庫管理システムでは，主記憶量と計算時間が削減されたが，人数管理システムでは増加した。

また，人数管理システムではNの値が大きいほど主記憶量と計算時間が大きいことがわかる。

1つのシステムの計算空間の詳細

人数管理システムのN=6, K=4の場合に各部分合成での合成と最小化の際の状態数と時間を計測

	最小化なし		最小化あり	
	状態数	時間 (ms)	状態数	時間 (ms)
1 回目合成	67	180	67	209
1 回目最小化	-	-	55	0
2 回目合成	107	82	92	62
2 回目最小化	-	-	83	0
3 回目合成	240	104	192	78
3 回目最小化	-	-	192	1
4 回目合成	368	107	307	86
4 回目最小化	-	-	281	3
5 回目合成	811	183	637	113
5 回目最小化	-	-	637	4
6 回目合成	1229	164	1004	170
6 回目最小化	-	-	929	13
7 回目合成	2674	234	2074	197
7 回目最小化	-	-	2074	39
8 回目合成	4014	319	3231	238
8 回目最小化	-	-	3015	108
9 回目合成	8649	584	6651	408
9 回目最小化	-	-	6651	334
10 回目合成	12879	711	10260	564
10 回目最小化	-	-	9639	1305
11 回目合成	60131	1946	46172	1576
11 回目最小化	-	-	45198	27096
12 回目合成	39665	1560	31322	1309

全段階で最小化のオーバーヘッドが合成より大きいわけではない

実験結果の分析

- 状態数と遷移数に関しては，いずれのシステムでも削減が見られ，LTSの最小化が状態数と遷移数の削減に効果的であることが確認できた。
- 主記憶量と計算時間に関しては，自動倉庫管理システムでは削減されたが人数管理システムでは増加したため，LTSの最小化が主記憶量と計算時間の削減に効果的であることは確認できなかった。

提案手法の限界

- LTSの最小化を加えたことによる追加の主記憶量と計算時間が無視できないほどに大きかった。

今後の展望

- LTSの最小化を用いる段階を，全段階ではなく，効果的な段階のみに適用することでLTSの最小化を効果的に用いる研究を進めていく。

今後の展望：最小化時間に注目

1つのシステムの計算空間の詳細

人数管理システムのN=6, K=4の場合に各部分合成での合成と最小化の際の状態数と時間を計測

	最小化なし		最小化あり	
	状態数	時間 (ms)	状態数	時間 (ms)
1 回目合成	67	180	67	209
1 回目最小化	-	-	55	0
2 回目合成	107	82	92	62
2 回目最小化	-	-	83	0
3 回目合成	240	104	192	78
3 回目最小化	-	-	192	1
4 回目合成	368	107	307	86
4 回目最小化	-	-	281	3
5 回目合成	811	183	637	113
5 回目最小化	-	-	637	4
6 回目合成	1229	164	1004	170
6 回目最小化	-	-	929	13
7 回目合成	2674	234	2074	197
7 回目最小化	-	-	2074	39
8 回目合成	4014	319	3231	238
8 回目最小化	-	-	3015	108
9 回目合成	8649	584	6651	408
9 回目最小化	-	-	6651	334
10 回目合成	12879	711	10260	564
10 回目最小化	-	-	9639	1305
11 回目合成	60131	1946	46172	1576
11 回目最小化	-	-	45198	27096
12 回目合成	39665	1560	31322	1309

全段階で最小化のオーバーヘッドが合成より大きいわけではない

→特定の段階では最小化を適用しない

今後の展望：最小化にかかる時間

提案手法の限界の解決のために、人数管理システムの各段階の部分合成の後の最小化の時間を計測

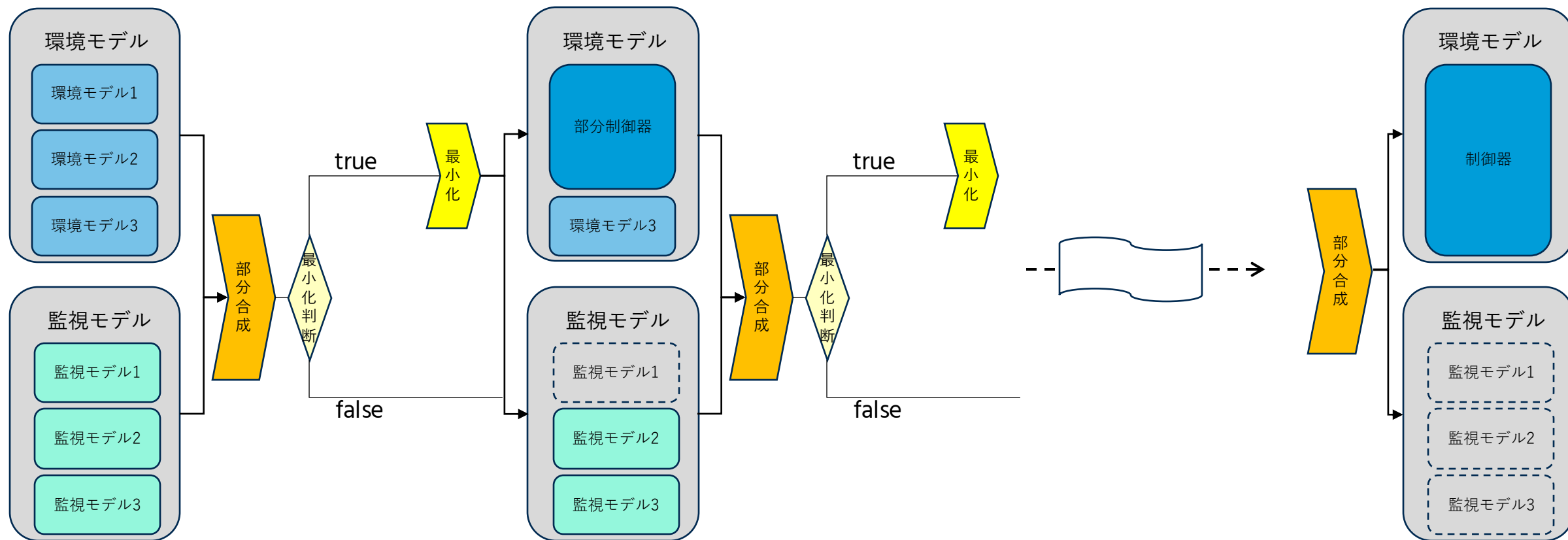
何回目の部分合成か	N=5, K=4	N=6, K=4
	最小化にかかる時間	
1回目	0	0
2回目	0	1
3回目	2	2
4回目	4	3
5回目	8	8
6回目	24	50
7回目	28	35
8回目	159	95
9回目	1,111	312
10回目	-	912
11回目	-	19,787

赤字の段階の最小化がボトルネックとなっていると考えられる

今後の展望：効果的なLTS最小化の適用

効果的な段階にのみ最小化を適用する

- 本研究の部分合成の後に**最小化判断**というプロセスを導入
- 最小化判断**：部分合成した後に最小化をするならtrue, しないならfalseを返す
- 現状の**最小化判断**では最小化にかかる時間が最も長い段階では最小化を適用しないことにした



今後の展望：結果(一部のみ)

N=5, K=4	最小化しない	提案手法(全段階で最小化)	効率的な最小化(9回目以外最小化)
Maximum States	15725	12350	12350
Maximum Transitions	27814	22603	22765
Maximum Memory (MB)	87.1	96.7	69.8
Execution Time (ms)	2152	3195	2236

N=6, K=4	最小化しない	提案手法(全段階で最小化)	効率的な最小化(11回目以外最小化)
Maximum States	60131	46172	46172
Maximum Transitions	106816	84865	85351
Maximum Memory (MB)	269.1	465.9	206.8
Execution Time (ms)	5879	25959	6941

最小化に時間がかかった段階では最小化をしない手法では、状態空間削減効果は今回の提案手法に劣るものの、主記憶量と実行時間が削減できていることがわかる。

→最小化に時間がかかるかどうかを入力モデルから判断する手法を研究していく

おわりに

背景：

離散事象システムを表すLTSを1つのLTSとして合成する技術として**離散制御器合成**がある。
しかし、離散制御器合成には状態空間の指数増加の課題がある。

提案手法：

離散制御器合成において段階的なLTSの最小化を提案した。
具体的には、**段階的制御器合成**の部分合成の後に**LTSを最小化**を適用した。

評価：

従来手法の段階的制御器合成との比較実験を行い、状態数と遷移数には削減効果があることがわかった。しかし、主記憶量と計算時間の削減効果には限界があることがわかった。

今後の展望：

効果的な段階にのみLTSの最小化を適用する段階的制御器合成手法の確立を構想している。
予備実験の結果、提案手法の限界への対処の可能性を確認できた。